OpenTelemetry, AI Observability, and Governance Audits

Reading Reference for LiteLLM, Langfuse, AgentOps, and LLM Governance

Sujeet Banerjee

Abstract

Modern AI applications are not single model calls. A production AI request may travel through an agent workflow, invoke one or more tools, retrieve documents from a vector database, access memory, call an LLM through a provider abstraction layer, and finally produce an answer.

This creates a difficult operational question:

> *If an AI system produces an incorrect, unsafe, expensive, or policy-violating result, can we reconstruct what happened?*

OpenTelemetry (OTel) provides a standardized way to describe and propagate telemetry across distributed systems. Langfuse builds AI-specific observability capabilities on top of OpenTelemetry concepts. LiteLLM can act as a model abstraction and routing layer through which model requests can be consistently observed and enriched with metadata.

This document explains the concepts of traces, spans, trace context, attributes, events, baggage, sessions, observations, metadata, and correlation identifiers. It then develops an architecture in which telemetry can become evidence for AI operations, debugging, evaluation, and governance audits.

The central argument is:

Observability → Execution Visibility → Evidence Collection → Evaluation and Controls → Auditability

Observability alone is not governance. However, without sufficiently structured execution evidence, meaningful technical governance and post-hoc auditing become substantially more difficult.

Contents

1 Relationship to the Bootcamp

This reading should be understood as connecting multiple days of the bootcamp rather than as an isolated OpenTelemetry topic.

Bootcamp Area	**Connection to this Reading**
Day 12: Agent Memory	Introduces the distinction between an execution trace and a longer-lived conversation or session.
Days 15–21: RAG	Retrieval, top-k documents, context construction, and RAG evaluation can become observable operations.
Day 22: LLMOps	Introduces production concerns beyond simply generating an answer.
Day 23: LiteLLM	Provides provider abstraction and a potential common observation point for model calls.
Day 24: Model Routing	Telemetry can show which model was selected, why, how much it cost, and how it performed.
Day 25: Langfuse	Introduces traces, spans, token usage, cost, and AI-specific observability.
Day 26: AgentOps	Tool calls, memory, retrieval, execution paths, failures, and latency become observable.
Day 27: Continuous Evaluation	Telemetry can be correlated with evaluation scores, prompt versions, and regression evidence.
Day 28: Final Capstone	All of these concepts meet in one end-to-end production-style architecture.

The bootcamp's final architecture is conceptually:

$$\text{n8n} + \text{Qdrant} + \text{LiteLLM} + \text{Model Providers} + \text{Langfuse}$$

The observability layer therefore should not be viewed as "something added after the application works." It is part of the production architecture.

2 The Core Problem: AI Systems are Execution Graphs

A simple chatbot may look like this:

$$\text{User} \rightarrow \text{LLM} \rightarrow \text{Response}$$

However, an agentic system may actually execute:

$$\text{User} \rightarrow \text{Workflow} \rightarrow \text{Agent} \rightarrow \begin{cases} \text{Memory Lookup} \\ \text{Retriever} \\ \text{Tool Call} \\ \text{LLM Call} \\ \text{Model Router} \end{cases} \rightarrow \text{Response}$$

A useful abstraction is to think of a request as producing an **execution graph**.
For a request R:

$$R = \{o_1, o_2, o_3, \ldots, o_n\}$$

where each o_i is an operation.
For example:

$$R = \begin{Bmatrix} \text{Receive Query}, \text{Load Memory}, \text{Retrieve Documents}, \\ \text{Build Prompt}, \text{LLM Generation}, \text{Tool Call}, \text{Final Response} \end{Bmatrix}$$

The central observability problem is therefore:

> How can all of these operations be connected so that they can later be understood as one coherent execution?

OpenTelemetry provides the fundamental answer through **traces**, **spans**, and **context propagation**.

3 What is OpenTelemetry?

OpenTelemetry, commonly abbreviated as OTel, is an open observability framework and specification.

It provides a standardized approach for collecting and propagating telemetry such as:

- traces,
- spans,
- metrics,
- logs,
- context,
- baggage,
- resource information.

The important architectural idea is:

$$\boxed{\text{Instrumentation} \rightarrow \text{Standard Telemetry} \rightarrow \text{Export} \rightarrow \text{Observability Backend}}$$

For example:

$$\text{Application} \rightarrow \text{OTel Instrumentation} \rightarrow \text{OTLP} \rightarrow \text{Langfuse / Collector / Other Backend}$$

OTel is not itself primarily an AI governance product.
Instead, it provides a common language for representing what happened inside a distributed system.

For AI applications, this is increasingly important because a single user request may involve multiple services, models, tools, databases, and execution environments.

4 The Trace

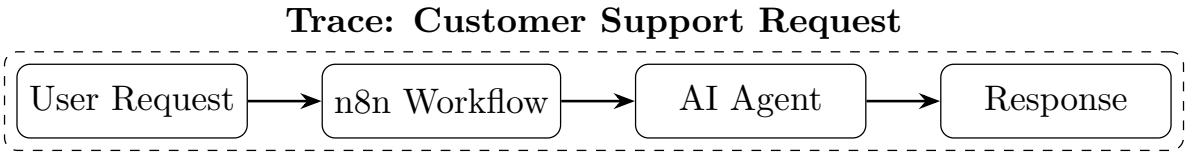
4.1 Concept

A **trace** represents the overall execution of a request or transaction.

For example:

"Answer the customer's question about the company's refund policy."

The complete lifecycle of handling this request can be represented as one trace.



The trace is therefore the answer to:

What happened during this particular end-to-end execution?

Each trace has an identifier:

`trace_id`

All operations belonging to the same distributed execution can be correlated using this identifier.

5 The Span

5.1 Concept

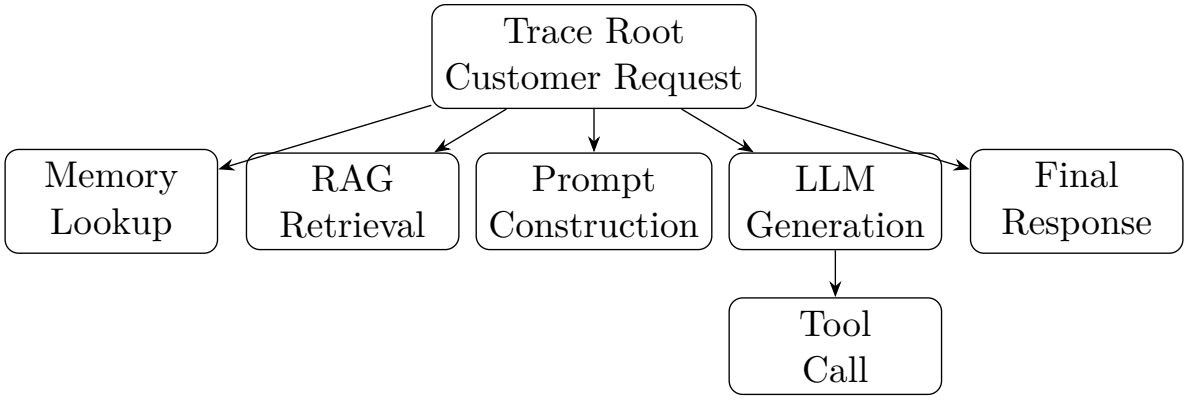
A **span** represents one unit of work within a trace.

Suppose an agent handles a customer request.

The trace might contain:

1. Receive Request
2. Retrieve Customer Memory
3. Retrieve Company Documents
4. Construct Prompt
5. Call LLM
6. Call CRM Tool
7. Generate Final Response

Each of these can be represented as a span.



Formally, a trace can be represented as:

$$T = \{S_1, S_2, \ldots, S_n\}$$

where each S_i is a span.
A span generally contains information such as:

$$S = (\text{name}, \text{span\_id}, \text{parent\_span\_id}, t_{start}, t_{end}, \text{attributes}, \text{events}, \text{status})$$

The duration of a span is:

$$\text{Latency}(S) = t_{end} - t_{start}$$

This immediately allows operational questions such as:

- Which operation was slow?
- Which model call failed?
- How long did retrieval take?
- Which tool caused the workflow to fail?

6 Parent–Child Relationships

Spans are normally connected through parent–child relationships.
For example:

$$S_{workflow} \rightarrow S_{agent} \rightarrow S_{retrieval}$$

or:

$$S_{agent} \rightarrow S_{generation} \rightarrow S_{tool}$$

This produces a trace tree.
The trace tree answers questions that ordinary logs often struggle to answer:

> "This LLM call took five seconds—but which user request caused it, which agent invoked it, and what happened before and after it?"

The parent–child structure preserves execution context.

7 Trace Context and Context Propagation

The trace must continue across process boundaries.

Consider:

$$\text{n8n} \rightarrow \text{LiteLLM Proxy} \rightarrow \text{Model Provider}$$

These may be separate processes or services.
Without context propagation, each system could produce isolated telemetry.
We might observe:

$$T_1 = \text{n8n request}$$

and separately:

$$T_2 = \text{LiteLLM model request}$$

and separately:

$$T_3 = \text{provider response}$$

But we would not necessarily know:

$$T_1 \leftrightarrow T_2 \leftrightarrow T_3$$

Context propagation attempts to preserve the relationship.
Conceptually:

$$\boxed{\text{Trace Context} = (\text{trace\_id}, \text{current span identity}, \text{other propagation state})}$$

When service A calls service B, relevant context can be injected into the outgoing request.
Service B extracts the context and creates a child span.
Thus:

$$S_A \rightarrow S_B$$

even though:

$$A \neq B$$

This is one of the fundamental reasons distributed tracing is useful.

8 SpanContext

A **SpanContext** contains the identifying information needed to propagate tracing relationships.

The important identifiers are:

- `trace_id`
- `span_id`

Conceptually:

$$\text{Trace} = \text{All spans sharing the same trace identifier}$$

while:

Span = One identifiable operation within that trace

For example:

```
trace_id = abc123

span_1 = Receive Request
span_2 = Retrieve Documents
span_3 = LLM Generation
span_4 = CRM Tool Call
```

All spans can belong to:

```
trace_id = abc123
```

This is what enables an observability platform to reconstruct the execution tree.

9 Attributes: Adding Meaning to Telemetry

A trace identifier tells us:

> "These operations belong together."

But it does not tell us:

> "What was the model?"

or:

> "Which prompt version was used?"

or:

> "Was this request from the production environment?"

This is where **attributes** become important.
An attribute is generally a key–value pair.
For example:

```
model = "gpt-4o"
environment = "production"
prompt_version = "v12"
tenant_id = "enterprise_42"
risk_level = "high"
```

Mathematically:

$$A(S) = \{(k_1, v_1), (k_2, v_2), \ldots, (k_n, v_n)\}$$

where $A(S)$ is the attribute set associated with span S.
Attributes transform raw telemetry into structured evidence.
Compare:

> LLM call failed.

with:

```
trace_id = abc123
model = gpt-4o
provider = openai
environment = production
prompt_version = v12
workflow = customer-support
tenant = enterprise_42
operation = refund-analysis
error = timeout
```

The second record is much more useful operationally and potentially more useful during an audit.

10 Events

A span describes an operation with duration.

An **event** can represent something that happened at a particular point in time during that operation.

For example, inside an agent execution span:

```
10:00:01 agent.started
10:00:02 retrieval.completed
10:00:03 tool.selected
10:00:04 tool.failed
10:00:05 fallback.triggered
```

Events can help answer:

- When did a failure occur?
- When did the agent change strategy?
- When was fallback activated?
- When was a policy control triggered?

For governance purposes, events can potentially represent important control points:

$$\text{Execution} \rightarrow \text{Policy Check Event} \rightarrow \text{Evaluation Event} \rightarrow \text{Fallback Event}$$

However, whether a platform records these automatically depends on the application's instrumentation.

11 Baggage

Baggage is contextual key–value information that can travel alongside distributed context.

Conceptually:

$$B = \{(k_1, v_1), (k_2, v_2), \ldots\}$$

Suppose an incoming request contains:

```
tenant_id = bank_42
application_id = loan-agent
environment = production
```

This contextual information may need to be available downstream.
For example:

$$\text{n8n} \rightarrow \text{LiteLLM} \rightarrow \text{Tool Service}$$

Baggage can conceptually help carry context across the chain.
However, an important distinction must be understood:

$$\boxed{\text{Baggage} \neq \text{Span Attributes}}$$

Baggage is propagated contextual information.
It must be explicitly used or copied into telemetry attributes if the implementation requires it to become queryable observation data.
Students should also understand the security implication:

> Do not treat baggage as a secure transport for sensitive data.

Sensitive information should be minimized, classified, and protected according to the application's security and privacy requirements.

12 Resources: What Produced the Telemetry?

A **resource** describes the entity producing telemetry.
Examples:

- application,
- service,
- container,
- virtual machine,
- Kubernetes workload,
- environment.

Conceptually:

$$\text{Telemetry Record} = \text{Execution Data} + \text{Resource Context}$$

For example:

```
service.name = litellm-proxy
deployment.environment = production
host.name = ai-gateway-01
service.version = 1.8.2
```

This is important because governance questions often require distinguishing:

$$\text{Development} \neq \text{Testing} \neq \text{Production}$$

A trace from an experimental environment should not automatically be interpreted in the same way as production evidence.

13 Sessions: A Different Concept from Traces

Students frequently confuse a **trace** with a **session**.

They are not the same.

13.1 Trace

A trace normally represents one end-to-end execution.

For example:

$$\text{One User Request} \rightarrow \text{One Trace}$$

13.2 Session

A session can represent a longer-running interaction.

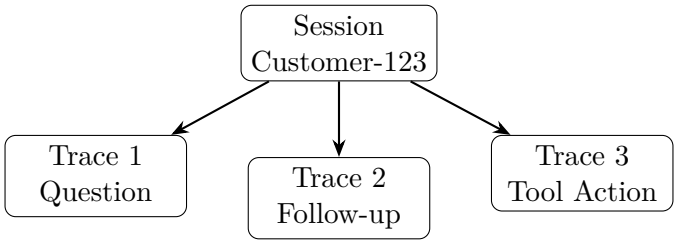
For example:

$$\text{Customer Conversation} = \{T_1, T_2, T_3, T_4, T_5\}$$

where:

$$T_i = \text{individual request trace}$$

Consider:



Therefore:

$$\boxed{\text{Session} = \text{Collection of related traces over time}}$$

This aligns conceptually with the bootcamp's Day 12 discussion of conversational state and session memory.

However, the following distinction is important:

$$\text{Session ID} \neq \text{Memory Store}$$

A session identifier helps correlate interactions.

Memory is actual state or information available to the agent.

14 AI-Specific Observability

Traditional distributed tracing might contain operations such as:

- HTTP request,
- database query,
- cache lookup,

- RPC call.

AI systems introduce additional operations:

- LLM inference,
- embeddings,
- vector retrieval,
- context construction,
- agent planning,
- memory retrieval,
- tool execution,
- evaluation.

Conceptually, a GenAI trace can therefore be represented as:

$$T = \{S_{workflow}, S_{memory}, S_{retrieval}, S_{prompt}, S_{generation}, S_{tool}, S_{evaluation}\}$$

Recent OpenTelemetry work includes evolving semantic conventions for GenAI operations.
The goal of semantic conventions is standardization.
Instead of every organization inventing:

```
my_model_name
ai_model
llm_name
used_model
```

a common semantic vocabulary can improve interoperability.

15 Why Semantic Conventions Matter

Suppose three systems record model information differently:

```
System A:
model = gpt-4o

System B:
llm_model = gpt-4o

System C:
selected_model = gpt-4o
```

Aggregating this data is inconvenient.
Semantic conventions attempt to standardize meaning.
The general principle is:

Standardized Names → Better Correlation → Better Aggregation → Better Interoperability

This becomes particularly valuable in a multi-vendor architecture.
For example:

$$\text{n8n} \rightarrow \text{LiteLLM} \rightarrow \text{OpenAI}$$

or:

$$\text{n8n} \rightarrow \text{LiteLLM} \rightarrow \text{Ollama}$$

A consistent telemetry vocabulary can make cross-provider analysis easier.

16 Langfuse and OpenTelemetry

Langfuse provides AI-specific observability capabilities while building its SDK architecture around OpenTelemetry concepts.

Conceptually:

$$\text{OTel Trace} \approx \text{Langfuse Trace}$$

and:

$$\text{OTel Span} \approx \text{Langfuse Observation}$$

Langfuse further specializes observations.
For example:

$$\text{Observation} \in \{\text{Span}, \text{Generation}, \text{Event}, \text{Tool}, \text{Retrieval}\}$$

An LLM generation is more specialized than a generic span because it may contain AI-specific information:

$$G = (\text{model}, \text{parameters}, \text{token usage}, \text{cost}, \text{latency}, \text{input}, \text{output})$$

This is one reason AI observability platforms provide more useful abstractions for LLM applications than generic APM systems alone.

17 Trace-Level Correlation in Langfuse

A Langfuse trace can contain shared information such as:

- user identifier,
- session identifier,
- metadata,
- tags,
- version,
- environment.

Conceptually:

$$T = (\text{trace\_id}, \text{user\_id}, \text{session\_id}, \text{metadata}, \text{tags}, \text{observations})$$

This enables questions such as:

Show all failed traces associated with:

```
environment = production
application = customer-support
prompt_version = v12
model = gpt-4o
```

Or:

Calculate the average cost per session for a particular workflow.

This is where metadata design becomes strategically important.

18 The LiteLLM Layer

LiteLLM provides a model abstraction layer.

Conceptually:

$$\text{Application} \rightarrow \text{LiteLLM} \rightarrow \begin{cases} \text{OpenAI} \\ \text{Anthropic} \\ \text{Ollama} \\ \text{Other Providers} \end{cases}$$

The application can interact with a common interface while LiteLLM manages provider-specific behavior.

For the bootcamp architecture:

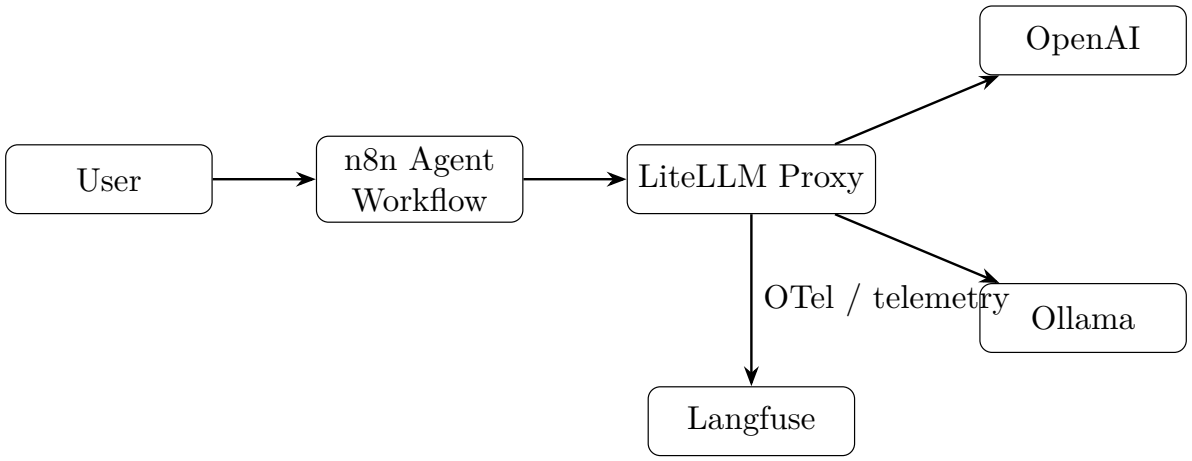
$$\text{n8n Agent} \rightarrow \text{LiteLLM Proxy} \rightarrow \text{Selected Model}$$

This makes LiteLLM strategically interesting for observability because it can become a common point through which model requests flow.

Instead of instrumenting every possible provider independently, the model gateway can become an important observation boundary.

19 LiteLLM as an Observability Boundary

Consider the following architecture:



The important design idea is:

$$\boxed{\text{Model Gateway} = \text{Potential Central Observation Point}}$$

LiteLLM can potentially expose information such as:

- selected model,
- provider,
- request timing,
- token usage,
- cost,
- errors,
- metadata supplied by the calling application.

This does not automatically mean LiteLLM observes the entire n8n workflow.
For full end-to-end visibility, instrumentation and correlation must extend across the relevant components.

20 Metadata: The Foundation of Correlation

The most important practical lesson for governance-oriented observability is:

Telemetry without meaningful metadata has limited audit value.

Suppose we only record:

```
model = gpt-4o
latency = 2.4 seconds
cost = $0.03
```

We know something about the model call.
But we may not know:

- Which application generated it?
- Which workflow generated it?
- Which prompt version was used?
- Which model routing policy selected the model?
- Was it development or production?
- Which evaluation suite was associated with the deployment?
- Which governance policy applied?

A stronger metadata model might be:

```
application.name = customer-support-agent
application.version = 2.4.1

environment = production

workflow.id = refund-workflow
workflow.version = 14

agent.id = support-agent
agent.version = 3.2
```

```
prompt.name = refund-analysis
prompt.version = v12

model.alias = complex-task
model.selected = gpt-4o
provider = openai

evaluation.dataset = refund-regression-set
evaluation.version = 5

governance.policy = customer-data-policy-v3
risk.classification = medium

session_id = session_123
tenant_id = tenant_456
```

The objective is not to record everything.

The objective is to record enough structured information to answer important questions later.

21 Metadata Dimensions for AI Governance

A useful metadata taxonomy is:

21.1 1. Identity

What system produced this event?

```
application.id
workflow.id
agent.id
service.name
```

21.2 2. Version

Which version was running?

```
application.version
workflow.version
prompt.version
agent.version
```

21.3 3. Model

Which AI system was used?

```
model.alias
model.name
provider
routing.policy
```

21.4 4. Context

What kind of execution was this?

```
environment
tenant_id
session_id
request_type
```

21.5 5. Governance

Which controls or policies were relevant?

```
risk.classification
policy.id
policy.version
approval.status
data.classification
```

21.6 6. Evaluation

What quality evidence is associated with this execution?

```
evaluation.dataset
evaluation.version
faithfulness.score
context_precision.score
pass_fail
```

This taxonomy connects directly to the bootcamp's progression from evaluation to RAGOps, AgentOps, PromptOps, ModelOps, and continuous evaluation.

22 A Complete Correlation Model

A robust AI system may need several identifiers.

For example:

$$\text{Session ID} \rightarrow \text{Conversation}$$

$$\text{Trace ID} \rightarrow \text{Single Execution}$$

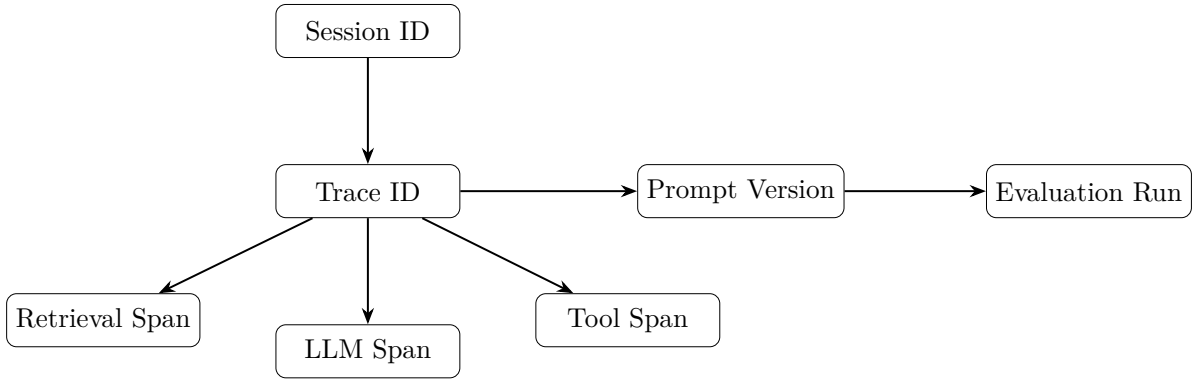
$$\text{Span ID} \rightarrow \text{Individual Operation}$$

$$\text{Prompt Version} \rightarrow \text{Prompt Artifact}$$

$$\text{Model Version/Alias} \rightarrow \text{Model Selection}$$

$$\text{Evaluation Run ID} \rightarrow \text{Quality Evidence}$$

These can be represented as a correlation graph:



This supports a governance-oriented question such as:

> "Show me all production executions of Agent A using Prompt Version 12 where the faithfulness score dropped below the approved threshold."

Without correlation identifiers and metadata, answering this may require manual forensic investigation.

23 Filtering

Filtering means selecting telemetry records based on attributes.

For example:

$$\text{environment} = \text{production}$$

or:

$$\text{model} = \text{gpt-4o}$$

or:

$$\text{prompt.version} = \text{v12}$$

A conceptual query:

```
environment = "production"
AND
workflow = "customer-support"
AND
prompt_version = "v12"
AND
error = true
```

Filtering is useful for:

- incident investigation,
- regression analysis,
- cost analysis,
- governance review,
- deployment comparison.

24 Aggregation

Aggregation means computing information across multiple executions.

Suppose N requests use model M.

Average latency:

$$\bar{L}_M = \frac{1}{N} \sum_{i=1}^{N} L_i$$

Total cost:

$$C_M = \sum_{i=1}^{N} c_i$$

Failure rate:

$$F_M = \frac{\text{Number of Failed Executions}}{\text{Total Executions}}$$

Average faithfulness:

$$\bar{F} = \frac{1}{N} \sum_{i=1}^{N} F_i$$

Now consider grouping by prompt version:

$$\bar{F}_{v11} \quad vs \quad \bar{F}_{v12}$$

This begins to connect observability with continuous evaluation.

25 From Traces to AI Governance Evidence

An AI audit may ask questions such as:

- What system made this decision?
- Which model was used?
- Which prompt version was active?
- What external information was retrieved?
- Were tools invoked?
- What controls were triggered?
- Was the system evaluated before deployment?
- Were quality thresholds met?
- Did the production system deviate from expected behavior?

A trace can potentially provide part of the answer.
Consider:

$$\text{Audit Question} \rightarrow \text{Required Evidence} \rightarrow \text{Telemetry Correlation}$$

For example:

Audit Question	Potential Technical Evidence
Which workflow executed?	Workflow ID and version metadata.
Which model responded?	Model and provider metadata.
What happened during execution?	Trace and span hierarchy.
Was retrieval involved?	Retrieval span and associated metadata.
Were external tools called?	Tool execution observations.
How expensive was execution?	Token usage and cost information.
Was the system evaluated?	Evaluation run and score metadata.
Did policy controls activate?	Policy-related events or attributes.

The critical word is **potential**.
Telemetry is not automatically sufficient evidence for every audit requirement.
Governance evidence may also require:

- access control records,
- approval workflows,
- model inventory,
- risk assessments,
- evaluation reports,
- policy documents,
- change management records,
- retention controls,
- data governance records.

26 Observability is Not Governance

This distinction is essential.

26.1 Observability asks

What happened?

26.2 Governance asks

Should this system exist, who is responsible for it, what controls apply, was it appropriately evaluated, and can compliance or accountability be demonstrated?

Therefore:

$$\boxed{\text{Telemetry} \neq \text{Governance}}$$

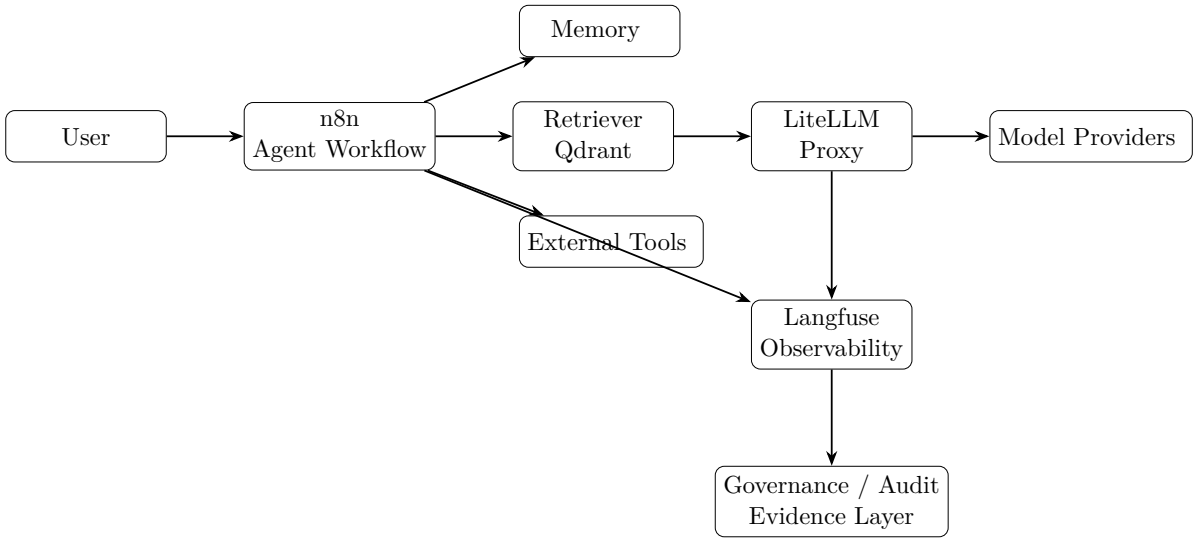
A better model is:

$$\boxed{\text{Governance Evidence} = f(\text{Telemetry}, \text{Policies}, \text{Evaluations}, \text{Approvals}, \text{Risk Controls}, \text{Provenance})}$$

Telemetry becomes one important evidence stream.

27 A Governance-Oriented Architecture

Consider the following architecture.



The observability system receives structured execution information.
The governance layer may then consume:

$$\begin{aligned}&\text{Trace Evidence}\\ &+\text{Evaluation Evidence}\\ &+\text{Version Metadata}\\ &+\text{Policy Metadata}\\ &+\text{Deployment Metadata}\end{aligned}$$

to create a stronger audit trail.

28 Example: Investigating a Bad AI Response

Suppose a customer receives an incorrect refund policy answer.
The trace might show:

$$T_{123}$$

with:

$$S_1 = \text{User Request}$$

$$S_2 = \text{Memory Retrieval}$$

$$S_3 = \text{RAG Retrieval}$$

$$S_4 = \text{Prompt Construction}$$

$$S_5 = \text{LLM Generation}$$

An investigator could potentially inspect:

1. Which documents were retrieved?
2. Was the correct document missing?
3. What prompt version was used?
4. Which model generated the answer?
5. Was a fallback model activated?
6. What evaluation score was associated with the release?

The failure could be classified as:

$$\text{Failure} \in \{\text{Retrieval Failure}, \text{Prompt Failure}, \text{Model Failure}, \text{Tool Failure}, \text{Memory Failure}, \text{Workflow Failure}\}$$

This is the connection between observability and systematic AgentOps debugging.

29 Example: RAG Audit Trail

For the bootcamp's enterprise knowledge assistant:

$$\text{Question} \rightarrow \text{Retriever} \rightarrow \text{Top-}k\text{ Documents} \rightarrow \text{LLM} \rightarrow \text{Answer}$$

An observable execution might contain:

```
Trace:
user_question

Span:
embedding_query

Span:
vector_search
top_k = 3

Span:
context_construction
retrieved_documents = [...]

Span:
llm_generation
model = ...
prompt_version = ...

Span:
evaluation
faithfulness = ...
context_precision = ...
```

This can support:

$$\text{Answer} \leftarrow \text{Model} \leftarrow \text{Prompt} \leftarrow \text{Retrieved Context} \leftarrow \text{Knowledge Source}$$

This begins to resemble an execution provenance chain.

30 Example: Model Routing Audit

Suppose the routing logic is:

$$\text{Simple Task} \rightarrow \text{Llama}$$

$$\text{Complex Task} \rightarrow \text{GPT-4o}$$

The trace should ideally make the routing decision observable.
For example:

```
routing.policy = complexity-router-v3
task.classification = complex
selected.model = gpt-4o
fallback.used = false
```

An audit or engineering investigation could ask:

> Did the system actually follow its intended routing policy?

This is different from merely recording:

```
model = gpt-4o
```

The routing decision itself is operationally relevant.

31 Example: PromptOps and Traceability

Day 27 introduces:

- prompt versioning,
- prompt testing,
- rollback.

A production trace should ideally be correlatable with a prompt artifact.
For example:

$$\text{Production Trace} \rightarrow \text{Prompt Name} + \text{Prompt Version}$$

Suppose:

$$P_{11} \rightarrow \text{Faithfulness} = 0.92$$

while:

$$P_{12} \rightarrow \text{Faithfulness} = 0.76$$

A regression becomes easier to detect if executions can be grouped by prompt version.
This leads to:

$$\boxed{\text{PromptOps} + \text{Observability} = \text{Operational Prompt Traceability}}$$

32 Connecting Evaluation to Observability

The bootcamp introduces RAGAS-style evaluation concepts such as:

- faithfulness,
- context precision,
- answer relevance,
- context recall.

These evaluation results should not necessarily remain isolated CSV values. Conceptually:

$$\text{Evaluation Result} \leftrightarrow \text{Prompt Version} \leftrightarrow \text{Model} \leftrightarrow \text{Dataset} \leftrightarrow \text{Deployment}$$

A useful evaluation record could therefore include:

```
evaluation.run_id
evaluation.dataset_version
prompt.version
model
faithfulness
context_precision
answer_relevance
pass_fail
```

This supports:

$$\text{CI/CD} \rightarrow \text{Evaluation} \rightarrow \text{Threshold} \rightarrow \text{Deployment Decision}$$

Later production telemetry can then be compared against pre-deployment evidence.

33 Continuous Evaluation and Governance

A one-time evaluation answers:

> "Did the system perform acceptably during this test?"

Continuous evaluation attempts to answer:

> "Is the system continuing to perform acceptably as prompts, models, data, and workflows change?"

A governance-oriented lifecycle is:

$$\text{Change} \rightarrow \text{Evaluation} \rightarrow \text{Pass/Fail} \rightarrow \text{Deploy} \rightarrow \text{Observe} \rightarrow \text{Detect Drift or Regression}$$

This is significantly stronger than:

$$\text{Build} \rightarrow \text{Deploy} \rightarrow \text{Hope}$$

34 A Possible AI Audit Evidence Model

A useful conceptual model is:

$$E = \{I, V, X, Q, C\}$$

where:

$$I = \text{Identity Evidence}$$

$$V = \text{Version Evidence}$$

$$X = \text{Execution Evidence}$$

$$Q = \text{Quality Evidence}$$

$$C = \text{Control Evidence}$$

Therefore:

$$\boxed{\text{AI Audit Evidence} = I + V + X + Q + C}$$

Examples:

34.1 Identity Evidence

- Application ID
- Agent ID
- Workflow ID

34.2 Version Evidence

- Prompt version
- Workflow version
- Model alias
- Application release

34.3 Execution Evidence

- Trace ID
- Span hierarchy
- Tool calls
- Retrieval operations

34.4 Quality Evidence

- Evaluation dataset
- Metric scores
- Pass/fail thresholds

34.5 Control Evidence

- Policy checks
- Guardrail events
- Approval status
- Risk classification

35 Important Limitations

Students should avoid making the following incorrect claims.

35.1 Incorrect Claim 1

"If we have traces, we are compliant."

False.
Traces may provide useful evidence, but compliance depends on the applicable legal, regulatory, organizational, and technical requirements.

35.2 Incorrect Claim 2

"OTel automatically captures every part of my AI system."

False.
Instrumentation and integration determine what is observed.
Uninstrumented components can remain invisible.

35.3 Incorrect Claim 3

"A trace proves why the model made a decision."

Usually too strong.
A trace can show execution steps, inputs, outputs, tools, and context, but this is not equivalent to a complete explanation of internal model reasoning.

35.4 Incorrect Claim 4

"More telemetry is always better."

False.
Telemetry can create:

- privacy risks,
- sensitive data exposure,
- storage costs,
- security risks,
- excessive operational noise.

The goal is:

$$\boxed{\text{Sufficient, Structured, Protected Evidence}}$$

not unlimited data collection.

36 Privacy and Sensitive Data

AI telemetry can contain:

- user prompts,
- model outputs,
- retrieved documents,
- tool parameters,
- identifiers.

Therefore:

$$\text{Observability Data} \subseteq \text{Potentially Sensitive Data}$$

Possible design controls include:

- data minimization,
- PII masking,
- selective content capture,
- hashing or pseudonymization,
- access controls,
- retention policies,
- environment separation.

This creates an important governance tension:

$$\text{More Detail} \rightarrow \text{Better Debugging}$$

but also potentially:

$$\text{More Detail} \rightarrow \text{Higher Privacy Exposure}$$

The engineering problem is therefore an optimization problem rather than a simple "capture everything" decision.

37 Recommended Metadata Contract

For a production AI platform, define a metadata contract.

For example:

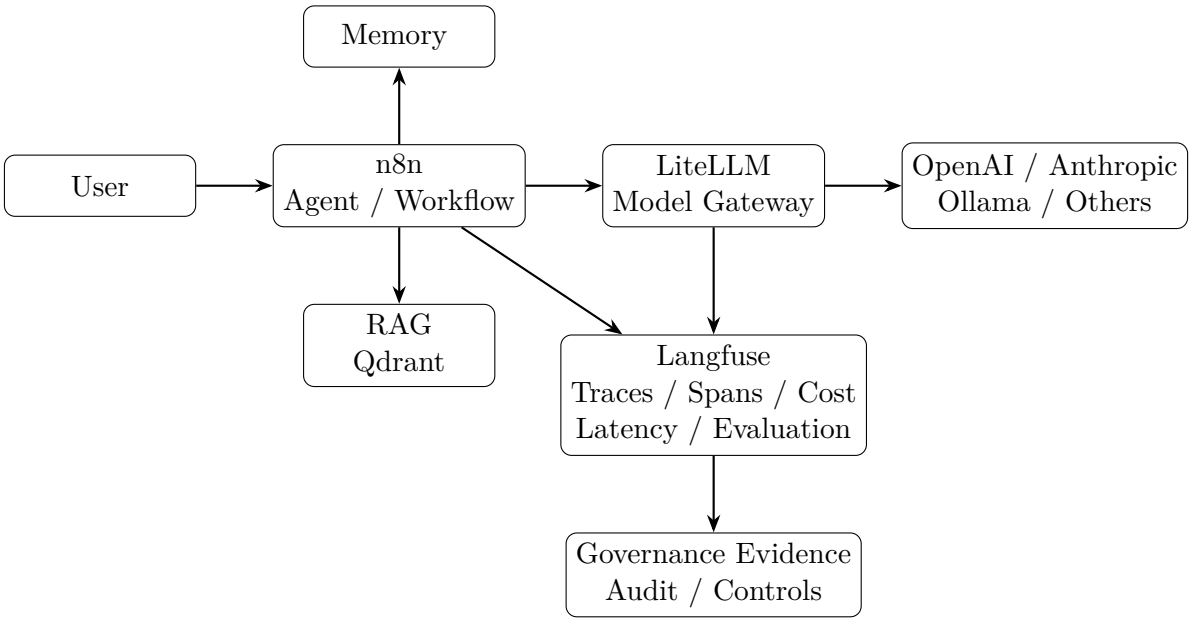
```
REQUIRED:

application.id
environment
workflow.id
workflow.version
trace_id

RECOMMENDED:
```

```
agent.id
agent.version
prompt.name
prompt.version
model.alias
model.name
provider

GOVERNANCE:

risk.classification
policy.id
policy.version
data.classification

EVALUATION:

evaluation.dataset
evaluation.version
evaluation.run_id
```

This should ideally become a documented contract rather than arbitrary metadata added by individual developers.

38 The Complete Bootcamp Ecosystem

The end-to-end ecosystem can be understood as:



The conceptual flow is:

Agentic Execution → Telemetry → Observability → Evaluation → Governance Evidence

39 Key Takeaways

Students should remember the following hierarchy.

Trace

The complete lifecycle of one distributed execution.

$$\text{One Request} \approx \text{One Trace}$$

Span

One operation inside that execution.

$$\text{Trace} = \sum \text{Spans}$$

conceptually speaking.

Context Propagation

The mechanism that helps preserve execution relationships across components.

$$\text{Service A} \rightarrow \text{Service B} \rightarrow \text{Service C}$$

while maintaining the execution relationship.

Attributes

Structured metadata describing operations.

$$\text{Span} + \text{Attributes} = \text{Meaningful Operational Record}$$

Session

A longer-lived grouping of multiple related traces.

$$\text{Session} = \{T_1, T_2, \ldots, T_n\}$$

LiteLLM

A model abstraction and gateway layer that can act as an important common observation point for model calls.

Langfuse

An AI observability platform that represents and analyzes AI execution using traces and OTel-based concepts, with specialized observations such as generations.

Governance

A broader discipline that can consume telemetry as evidence but also requires policies, controls, evaluation, accountability, and appropriate evidence management.

40 Discussion Questions

1. If a trace records every model call but does not record the prompt version, what audit question becomes difficult to answer?
2. What is the difference between a session ID and a trace ID?
3. Why is a model gateway such as LiteLLM a strategically useful observation point?
4. If the final answer is wrong, how can spans help distinguish between:
   - retrieval failure,
   - prompt failure,
   - model failure,
   - tool failure?
5. Should all prompts and model outputs be stored in observability data? What are the privacy trade-offs?
6. How would you correlate:

$$\text{Prompt Version} \rightarrow \text{Evaluation Run} \rightarrow \text{Deployment} \rightarrow \text{Production Trace}$$

7. Can execution observability demonstrate that an AI system is trustworthy, or does it only provide evidence about what happened?
8. What additional controls would be required to convert an observability platform into a stronger AI governance evidence platform?

41 Suggested Hands-On Exercise

Extend the bootcamp's LiteLLM + Langfuse architecture.

For every request, define:

```
application.id
workflow.id
workflow.version
environment
session_id
prompt.name
prompt.version
model.alias
risk.classification
```

Then perform the following analysis:

1. Run five requests using Prompt Version 1.
2. Run five requests using Prompt Version 2.
3. Record traces in Langfuse.
4. Compare:
   - latency,
   - token usage,

- cost,
- answer quality.

5. Add an evaluation score.
6. Filter traces by prompt version.
7. Identify whether Version 2 improved or degraded performance.

The objective is to demonstrate:

Prompt Change → Evaluation Evidence → Production Observation

42 Recommended Reading and References

Primary Technical References

1. OpenTelemetry Specification.
https://opentelemetry.io/docs/specs/otel/
2. OpenTelemetry Tracing API and Trace Concepts.
https://opentelemetry.io/docs/specs/otel/trace/api/
3. OpenTelemetry Context and Baggage Concepts.
https://opentelemetry.io/docs/concepts/signals/baggage/
4. OpenTelemetry Semantic Conventions.
https://opentelemetry.io/docs/concepts/semantic-conventions/
5. OpenTelemetry GenAI Semantic Conventions Repository.
https://github.com/open-telemetry/semantic-conventions-genai
6. Langfuse Observability SDK Overview and OpenTelemetry Foundation.
https://langfuse.com/docs/observability/sdk/overview
7. Langfuse Data Model and Trace/Observation Concepts.
https://langfuse.com/docs/observability/data-model
8. Langfuse LiteLLM SDK Integration.
https://langfuse.com/integrations/frameworks/litellm-sdk
9. LiteLLM Documentation.
https://docs.litellm.ai/

Recent Research: AI Observability, Provenance, and Governance

The following recent work is particularly relevant because it moves the discussion beyond traditional monitoring toward execution provenance, evidence tracing, and telemetry-driven governance.

1. Y. Wang et al. (2026).
From Agent Traces to Trust: Evidence Tracing and Execution Provenance in LLM Agents.
arXiv.
This survey is especially relevant to the distinction between simply recording execution traces and establishing provenance relationships among retrieval, tools, memory, intermediate actions, and final outputs.
https://arxiv.org/abs/2606.04990

2. N. K. Naik et al. (2026).
Traccia: An OpenTelemetry-Based Governance Platform for AI Systems.
arXiv.
A recent proposal explicitly exploring OTel-based AI governance, telemetry evidence, execution lineage, and compliance evidence generation.
https://arxiv.org/abs/2607.14309

3. E. Bandara et al. (2026).
AI Trust OS – A Continuous Governance Framework for Autonomous AI Observability and Zero-Trust Compliance in Enterprise Environments.
arXiv.
Presents a telemetry-first perspective in which continuous observation contributes to evidence-based governance.
https://arxiv.org/abs/2604.04749

4. I. Wicaksono et al. (2025).
Mind the Gap: Evaluating Model- and Agentic-Level Vulnerabilities in LLMs with Action Graphs.
NeurIPS 2025 LLM Evaluation Workshop.
Relevant to observability-based analysis of agent execution paths and agent-specific vulnerabilities.
https://openreview.net/forum?id=Bo9FcSnrVP

5. C. Yueh-Han et al. (2025).
Monitoring LLM Agents for Sequentially Contextual Harm.
ICLR 2025 Bi-Align Workshop.
Particularly useful for understanding why isolated observation of individual actions may fail to detect harmful behavior that emerges only across a sequence of actions.
https://openreview.net/forum?id=PGsM81SWHt

6. M. E. Mazzolenis and R. Zhang (2025).
Traxgen: Ground-Truth Trajectory Generation for AI Agent Evaluation.
NeurIPS 2025 Workshop.
Relevant to trajectory-level evaluation and the shift from final-answer evaluation toward execution-level evaluation.
https://openreview.net/forum?id=pERJAy5kI1

Important Research Caution

Some of the most recent governance papers in this area are preprints or workshop publications. Students should therefore distinguish between:

Peer-reviewed consensus

and:

Emerging architectural proposals

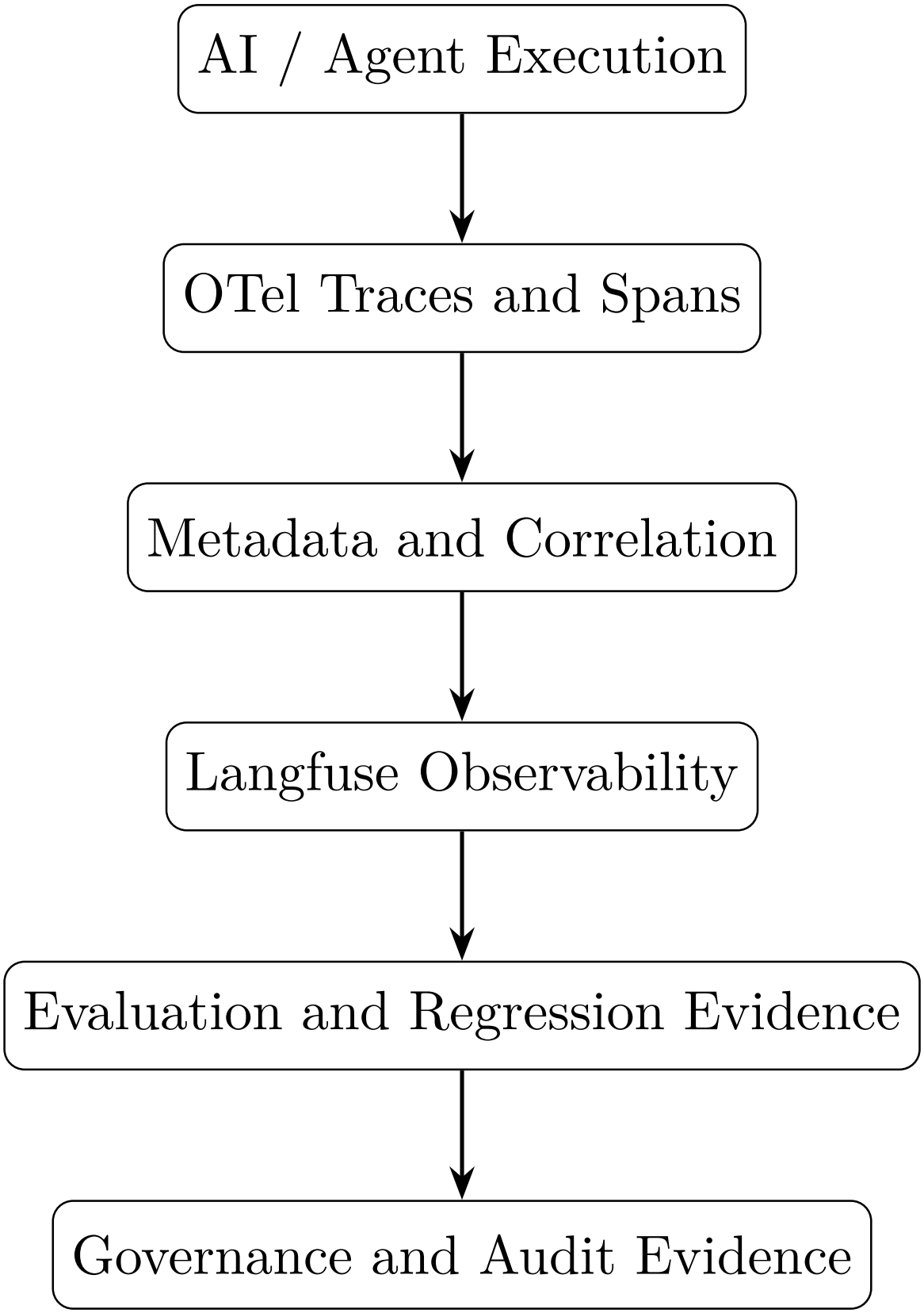
The value of these recent papers is that they identify an increasingly important direction:

AI Governance is moving toward continuous, execution-aware evidence

rather than relying only on static policy documents or one-time pre-deployment evaluations.

43 Final Mental Model

The complete mental model for this part of the bootcamp is:



The most important conclusion is:

If we cannot reconstruct how an AI system executed,

then:

debugging, evaluation, accountability, and auditing become much harder.

But the inverse is also important:

Being able to reconstruct execution does not, by itself, make the system governed.

A mature AI system therefore needs both:

Operational Visibility + Governance Controls

with structured telemetry serving as an important bridge between the two.